

总结、模型、数据、训推、实验（总分）

总结（背景+关键技术）

长程导航任务里如何进行长视频理解和长时基于对导航大模型带来了一些挑战，为了让模型具有长时记忆和在线推理，作者团队提出了SlowFast Context Modeling，Long term Memory做时空上的token pruning，而short term context用于推理，所有历史toke通过kv-cache高效服用，实现VLA的流式版本；训练过程中通过多元协同训练，使用多模态图文数据，保持模型对于多模型数据的泛化，在一个相对的合适的配比下完成了模型的训练

模型细节（Model）



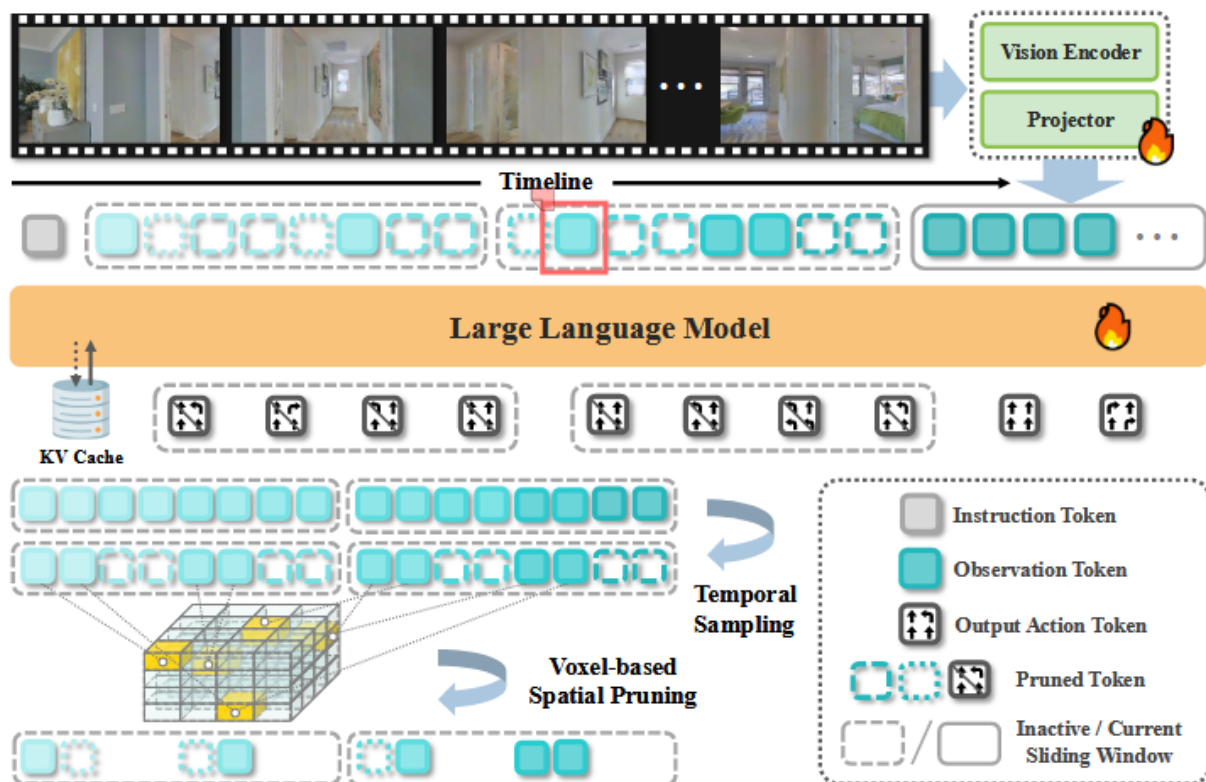


Figure 1: **Framework of StreamVLN.** The input consists of a language instruction and a stream of RGB images. Each navigation episode is framed as a multi-turn dialogue, where the agent continually queries for the next actions. **To support long-horizon reasoning while maintaining a manageable context size and low latency, we adopt a fixed-size sliding window to retain recent dialogue history.** The context in inactive windows is updated by token pruning to to reduce memory overhead.

StreamVLN is built on LLaVA-Video as the foundational Video-LLM, extended specifically for Vision-and-Language Navigation (VLN) tasks. The model architecture consists of:

主要组件

StreamVLN

- ├── LLaVA-Video-7B (基础)
- | └── 语言模型: Qwen2-7B-Instruct
- | └── 视觉编码器: Google SigLIP SO400M
- | └── 多模态投影器: mlp2x_gelu
- └── StreamVLN 扩展 (Extensions)
 - ├── 面向 VLN 的定制改动
 - ├── SlowFast 上下文建模
 - ├── 记忆 Token 机制
 - └── 动作生成头 (Action head)

- **StreamVLNModel** (`streamvln/model/stream_video_vln.py:14`) : 在 LLaVA-Video 基础上扩展的核心模型
 - 基于 **LLaVA-Video 7B** (语言模型为 **Qwen2-7B**) , 扩展为「视觉-语言-动作」一体化的在线流式对话式导航模型。
 - 代码参考: **LLaVA-VL / LLaVA-NeXT** (包含 LLaVA-Video 7B/72B 的发布与脚本, 见 GitHub)
 - 7B 模型卡/权重 (示例) : **Imms-lab / LLaVA-Video-7B-Qwen2** (基于 Qwen2, 最多 64 帧, 见 Hugging Face)
 - 论文: arXiv: <https://arxiv.org/pdf/2410.02713>
- **StreamVLNForCausalLM** (`streamvln/model/stream_video_vln.py:32`) : 用于训练/推理的因果语言模型封装
 - `StreamVLNModel` 是多模态编码核心; `StreamVLNForCausalLM` 作为完整训练/推理封装, 增加语言建模能力, 并提供端到端接口。
- **视觉塔 / 视觉编码器 (Vision Tower/Encoder)** : 支持多种架构 (如 SigLIP、EVA-CLIP 等) ; 默认为 Google **SigLIP SO400M** (SOTA 视觉编码器) 。
- **多模态投影器 (Multi-modal Projector)** : 将视觉特征映射到语言空间; 采用 **基于 MLP 的投影器** 以实现视-语对齐。
- **语言模型 (Language Model)** : 基于 **Qwen2** 的主干, 用于驱动动作生成。

Model Size:

- **总参数量**: 约 **7B (Qwen2-7B) + 视觉编码器 + 投影器**
- **训练**: 以 **LLaVA-Video-7B** 的预训练检查点为起点
- **微调**: 在 **VLN 特定任务与数据集** 上进一步训练

Streaming Mechanisms and SlowFast Context Modeling

1. Fast-Streaming Dialogue Context (快通道: 对话滑窗 + KV Cache 复用, 保证动作生成的低延迟。)

- Sliding-window KV cache (`stream_video_vln.py:396-407`) for efficient real-time processing
- Token-level streaming with incremental input embedding accumulation
- Environment-level caching to maintain dialogue state across navigation steps

2. Slow-Updating Memory System (慢通道: 历史视觉记忆 (保留高分辨率 token) , 配合 “体素空间 (3D-aware) token剪枝” 控制记忆增长与稳定解码速度。)

- Memory tokens (`MEMORY_TOKEN_INDEX`) for long-term context preservation
- 2D spatial pooling (`get_2dPool` method) for efficient memory compression

- 将视频帧的图像 patch 反投影到共享 3D 空间，离散为体素；同一持续时间窗口若多个 token 落同一体素，仅保留最新观测的 token（算法 1）。
- 论文给出伪代码（Voxel-Based Spatial Pruning）与门限/stride 的掩码生成过程。
- 推断期应用剪枝平均减少约 20% 输入 token，并在 R2R/RxR 上带来 ~+1% SR/SPL 增益。
- History management with configurable num_history parameter
- Token pruning strategies to handle long sequences efficiently

3. Multi-Modal Input Processing

- RGB-D encoding: Combined color and depth information
- Pose integration: Camera poses for spatial awareness
- Temporal batching: Multiple view processing with num_future_steps
- Memory feature extraction from historical observations

Key Technical Innovations

1. Interleaved Vision-Language-Action Modeling

a. Multi-turn dialogue format for continuous navigation

- i. 每轮对话 $d_i = (o_i, a_i)$ ，输出动作条件于当前观测与历史上下文；滑窗内 KV 可复用，窗外记忆经采样+剪枝压缩

b. Action tokenization with specialized navigation vocabulary

c. Real-time response generation with streaming inference

2. Efficient Context Management

代码块

```
1  # Memory token handling in encode_rgbd
2  if start_idx == 0:
3      memory_features.append(None)
4  else:
5      history_idx = self.model.num_history
6      his_image_feature = image_features[b, :history_idx]
7      memory_features.append(his_image_feature.flatten(0,1).unsqueeze(0))
```

3. Streaming Inference Pipeline

- Incremental input processing with cache management
- KV cache optimization for long sequences
- Multi-environment parallelization support

Action Generation and Control

Navigation Action Space

代码块

```
1  actions2idx = {
2      'STOP': [0],
3      "↑": [1],      # MOVE FORWARD 25cm
4      "←": [2],      # TURN LEFT 15°
5      "→": [3]       # TURN RIGHT 15°
6  }
```

Streaming Action Generation:

- Real-time inference with continuous video input
- Multi-step prediction with sequence generation
- Environment state tracking through conversation memory

Implementation Details

Memory Efficiency:

- **Token 压缩（2D 空间池化）**：对视频/图像特征在空间维度做池化与降采样，将长序列中冗余的空间信息压缩为更紧凑的 token，降低显存占用与计算量。
- **KV Cache 管理（长序列）**：对注意力的 Key/Value 缓存按时间分段、滑窗或打分保留（重要帧优先），在保持上下文连贯的同时控制缓存规模。
- **梯度检查点（Gradient Checkpointing）**：以计算换内存的方式在反向传播时重算部分前向过程，显著减少峰值显存。

Distributed Training:

- **多卡/多机（SLURM 调度）**：通过 SLURM 启动与管理集群任务，支持数据并行与（可选）模型/流水并行。

- **DeepSpeed 集成**：利用 ZeRO 优化器、参数/优化器状态分片与高效通信，加速大模型训练并扩大可训练批次。
- **混合精度 (bfloat16)**：默认使用 bfloat16，以更稳定的数值范围获得与 FP16 相近的吞吐，降低显存占用。

Evaluation Framework:

- **Habitat-sim 集成**：在逼真的 3D 模拟环境中进行在线流式导航评测，覆盖多场景与跨楼层探索等任务形态。
- **多指标评测**：报告 **Success Rate**、**SPL** (Success weighted by Path Length)、**Navigation Error** (终点误差) 等核心指标。
- **真实机部署 (Unitree Go2)**：提供在 Unitree Go2 等实机平台上的端到端部署与验证路径，打通“仿真—实机”闭环。

Technical Advantages

- **实时性**：流式 (Streaming) 架构边看边算，支持在线决策与动作输出，满足实时导航需求。
- **内存效率**：SlowFast 上下文建模配合 token 压缩与 KV Cache 管理，稳健处理长时序视频而不过度占用显存。
- **多模态理解**：统一“视觉—语言—动作”建模，既能理解自然语言指令，也能基于视觉上下文生成可执行动作。
- **可扩展性**：分布式训练与 DeepSpeed 支持大规模数据与更长序列，便于快速迭代与大样本学习。
- **灵活性**：模块化配置可适配不同视觉编码器、环境 (Habitat 场景/真实场景) 与机器人本体 (如 Go2) 。

This analysis reveals StreamVLN as a sophisticated streaming VLN system that efficiently combines real-time video processing with long-term memory management, enabling robust autonomous navigation through its innovative SlowFast context modeling architecture.

细节记录

resample是什么

代码块

```
1 Vision Resampler 是多模态模型中的一个关键组件，位于视觉编码器 (Vision
2 Tower) 和多模态投影器 (MM Projector) 之间，用于对视觉特征进行重采样和优化。
3
```



```
4  架构位置
5
6  原始图像 → Vision Tower (SigLIP) → Vision Resampler → MM Projector →
  Language Model
7      ↓           ↓           ↓           ↓           ↓
8  RGB像素      视觉特征提取      特征重采样/压缩      多模态融合      语言理解/
  生成
9
10 主要功能
11
12 1. 特征维度调整:
13 - 将视觉编码器输出的高维度特征调整为适合语言模型处理的维度
14 - 支持不同的输出通道数配置
15
16 2. 空间信息压缩:
17 - 减少视觉token的数量, 降低计算复杂度
18 - 保留重要的空间信息
```

数据细节 (Data)

场景与基准集 (VLN-CE 体系)

场景资源

1. MP3D (用于 R2R、RxR、EnvDrop) : 放到 `data/scene_datasets/mp3d/` 。 [GitHub](#)
2. HM3D (用于 ScaleVLN 子集) : 放到 `data/scene_datasets/hm3d/train/` 。 [GitHub](#)

VLN-CE episodes (按目录重命名解压到 `data/datasets/`)

1. R2R → `r2r/` ; RxR → `rxr/` ; EnvDrop → `envdrop/` ; ScaleVLN 子集 (约15万条, 已转 VLN-CE 格式) → `scalevln/` 。 [GitHub](#)

r2r的格式

github.com/jacobkrantz/VLN-CE (有不同的版本的)

<https://jacobkrantz.github.io/vlnce/>

代码块

```
1  Top-level Structure
2
3  - episodes: List of 10,819 navigation episodes
4  - instruction_vocab: Vocabulary dictionary with 2,711 words
```

Individual Episode Structure

Each episode contains:

- episode_id: Unique identifier (integer)
- trajectory_id: Trajectory identifier (integer)
- scene_id: 3D scene identifier (string, format: "mp3d/{scene_id}/{scene_id}.glb")
- start_position: Starting coordinates [x, y, z] (list of floats)
- start_rotation: Starting orientation [x, y, z, w] (list of floats)
- goals: Target position with radius (list of dict with 'position' and 'radius')
- info: Additional metadata including 'geodesic_distance'
- instruction: Navigation instruction containing:
 - instruction_text: Natural language instruction string
 - instruction_tokens: Tokenized instruction using vocabulary indices
- reference_path: Optimal path coordinates (list of [x, y, z] positions)

Instruction Vocabulary

- word_list: Complete vocabulary of 2,711 words
- word2idx_dict: Word-to-index mapping for tokenization
- Contains navigation-specific terms, room names, furniture, directions, etc.

Sample Data

- Instruction: "Go around the right side of the center unit and stop by the right side doorway with the dining table and mirror in it."
- Start position: [-16.27, 0.15, 0.72]
- Goal position: [-12.34, 0.15, 4.21]
- Geodesic distance: 6.43 meters
- Reference path: 3 waypoints for optimal navigation

This structure supports vision-and-language navigation tasks with 3D environment simulation, natural language instructions, and ground-truth paths for training and evaluation.

截图表示:


```

4         'episode_id': 1,
5         'trajectory_id': 4,
6         'scene_id': 'mp3d/7y3sRwLe3Va/7y3sRwLe3Va.glb',
7         'instruction': {
8             'instruction_text': 'Go around the right side...',
9             'instruction_tokens': [982, 141, 2202, ..., 0, 0, 0]
10        },
11        'start_position': [-16.267200469970703, 0.1518409252166748,
12        0.7207760214805603],
13        'start_rotation': [0.0, 0.0007963267107332633, 0.0,
14        0.9999996829318346],
15        'goals': [
16            {
17                'position': [-12.337400436401367, 0.1518409252166748,
18                4.213699817657471],
19                'radius': 3.0
20            }
21        ],
22        'reference_path': [
23            [-16.267200469970703, 0.1518409252166748, 0.7207760214805603],
24            [-16.284099578857422, 0.1518409252166748, 2.4123699665069580],
25            ...,
26            [-13.907199859619140, 0.1518409252166748, 4.2282099723815920],
27        ],
28        'info': {'geodesic_distance': 6.425291538238525},
29    },
30    ...
31],
32'instruction_vocab': [
33    'word_list': [..., 'orchids', 'order', 'orient', ...],
34    'word2idx_dict': {
35        ...,
36        'orchids': 1505,
37        'order': 1506,
38        'orient': 1507,
39        ...
40    },
41    'itos': [..., 'orchids', 'order', 'orient', ...],
42    'stoi': {
43        ...,
44        'orchids': 1505,
45        'order': 1506,
46        'orient': 1507,
47        ...
48    },
49    'num_vocab': 2504,
50    'UNK_INDEX': 1,

```

```
48         'PAD_INDEX': 0,  
49     ]  
50 }
```

rxr的格式

<https://github.com/google-research-datasets/RxR>

代码块

```
1     The RxR (Room-to-Room Extended) dataset in the data/datasets/rxr/train/  
    directory contains three  
2     main files with a total size of 187 MB:  
3  
4     1. train_follower.json.gz (61.0 MB)  
5     - Structure: Dictionary with single key 'episodes' containing list of 58,752  
    episodes  
6     - Content: Follower agent navigation episodes  
7     - Key Features:  
8         - Multi-lingual instructions (English, Hindi, Telugu)  
9         - Detailed timing information in instructions  
10        - MP3D scene environments  
11        - Reference paths for navigation  
12  
13    2. train_guide_gt.json.gz (73.4 MB)  
14    - Structure: Dictionary with 60,300 entries keyed by instruction IDs  
15    - Content: Ground truth expert demonstrations  
16    - Key Features:  
17        - Contains locations, actions, forward_steps  
18        - Expert trajectory data  
19        - Action sequences for navigation  
20  
21    3. train_guide.json.gz (52.7 MB)  
22    - Structure: Dictionary with single key 'episodes' containing list of 60,300  
    episodes  
23    - Content: Guide agent navigation episodes  
24    - Key Features:  
25        - Similar structure to follower but for guide navigation  
26        - Contains trajectory information and role data  
27  
28    Individual Episode Structure  
29    Each episode contains:  
30    - episode_id: Unique identifier  
31    - trajectory_id: Trajectory identifier  
32    - scene_id: MP3D scene identifier  
33    - instruction: Rich instruction data including:  
34        - instruction_id: Unique instruction identifier
```

- **instruction_text**: Natural language instruction
- **language**: Language code (en-US, hi-IN, en-IN, te-IN)
- **annotator_id**: Annotator information
- **edit_distance**: Edit distance metrics
- **timed_instruction**: Time-segmented instruction (54 segments average)
- **start_position**: Starting coordinates [x, y, z]
- **start_rotation**: Starting orientation quaternion
- **goals**: Target positions with radius
- **reference_path**: Optimal navigation path
- **info**: Additional metadata including role information

Key Statistics

- **Total episodes**: 58,752 (follower) + 60,300 (guide) = 119,052 total
- **Languages**: English (US), English (India), Hindi, Telugu
- **Scenes**: MP3D environments with ~53 unique scenes in sample
- **Path lengths**: 1-64 waypoints (average ~11-12)
- **Instruction length**: 13-339 words (average ~76 words)

Unique Features

1. **Multi-lingual support**: Instructions in multiple Indian languages
2. **Dual agent perspective**: Both follower and guide agent data
3. **Timing information**: Detailed temporal segmentation of instructions
4. **Expert demonstrations**: Ground truth action sequences
5. **Cultural adaptation**: Different language variants for the same navigation tasks

The RxR dataset represents a significant extension of the original R2R dataset with multilingual

capabilities and more detailed annotation for vision-and-language navigation research.

代码块

RxR Dataset File Differences

1. train_follower.json.gz (61.0 MB)

- Purpose: Navigation episodes for the follower agent
- Structure: List of 58,752 episodes under 'episodes' key
- Content:
 - Agent that follows navigation instructions
 - Multi-lingual instructions with timing information
 - Reference paths showing optimal routes
 - Start positions, goals, and scene data
- Key Fields: episode_id, instruction, start_position, reference_path, goals

14 2. train_guide_gt.json.gz (73.4 MB)

15

16 - Purpose: Ground truth expert demonstrations

17 - Structure: Dictionary with 60,300 entries keyed by instruction IDs

18 - Content:

19 - Expert demonstration data (not full episodes)

20 - locations: Precise position coordinates

21 - actions: Action sequences taken by expert

22 - forward_steps: Number of forward movements

23 - No natural language instructions - just expert paths

24 - Use Case: Training agents to mimic expert navigation behavior

25

26 3. train_guide.json.gz (52.7 MB)

27

28 - Purpose: Navigation episodes for the guide agent

29 - Structure: List of 60,300 episodes under 'episodes' key

30 - Content:

31 - Agent that gives navigation guidance

32 - Similar to follower but from guide perspective

33 - Contains role information in metadata

34 - Multi-lingual instructions

35 - Paths and navigation data

36 - Key Fields: episode_id, trajectory_id, instruction, path, info['role']

37

38 Key Differences Summary

39

40	Aspect	train_follower	train_guide_gt	
	train_guide			
41	-----/-----/-----/-----			
	-----/			
42	Agent Type	Instruction follower	Expert demonstrator	
	Instruction guide			
43	Data Format	Full episodes	Expert trajectories	Full
	episodes			
44	Instructions	✓ Multi-lingual	✗ None	✓ Multi-
	lingual			
45	Navigation Paths	✓ Reference paths	✓ Expert actions	✓ Agent
	paths			
46	Structure	List of episodes	Dict by instruction ID	List of
	episodes			
47	Timing Info	✓ Detailed timing	✗ None	✓
	Available			
48	Total Items	58,752 episodes	60,300 entries	60,300
	episodes			

49

50 Usage in Training

51 - train_follower: Train agents to follow navigation instructions

```
52 - train_guide_gt: Train agents to mimic expert navigation patterns
53 - train_guide: Train agents to provide navigation guidance
54
55 The RxR dataset provides a complete multi-agent, multilingual navigation
framework with both
56 instruction-following and expert-demonstration capabilities.
```

envdrop分析

代码块

```
1 Top-level Structure
2
3 - episodes: List of 146,304 navigation episodes (much larger than R2R)
4 - instruction_vocab: Vocabulary dictionary with 2,504 words
5
6 Individual Episode Structure
7
8 Each episode contains the same structure as R2R:
9 - episode_id: Unique identifier (integer)
10 - trajectory_id: Trajectory identifier (integer)
11 - scene_id: 3D scene identifier (string, format:
"mp3d/{scene_id}/{scene_id}.glb")
12 - start_position: Starting coordinates [x, y, z] (list of floats)
13 - start_rotation: Starting orientation [x, y, z, w] (list of floats)
14 - goals: Target position with radius (list of dict with 'position' and
'radius')
15 - info: Additional metadata including 'geodesic_distance'
16 - instruction: Navigation instruction containing:
17 - instruction_text: Natural language instruction string
18 - instruction_tokens: Tokenized instruction using vocabulary indices (with
padding)
19 - reference_path: Optimal path coordinates (list of [x, y, z] positions)
20
21 Instruction Vocabulary
22
23 - word_list: Complete vocabulary of 2,504 words
24 - word2idx_dict: Word-to-index mapping
25 - stoi: String-to-index mapping (same as word2idx_dict)
26 - itos: Index-to-string mapping
27 - num_vocab: Vocabulary size (2,504)
28 - UNK_INDEX: Unknown token index (1)
29 - PAD_INDEX: Padding token index (0)
30
31 Sample Data
32
```

```

33     - Instruction: "walk past the table and chairs and turn right . walk up the
34       top of the stairs ."
35     - Start position: [2.74, 0.09, -9.06]
36     - Goal position: [4.86, 2.61, -10.89] (note: height change indicates multi-
37       level navigation)
38     - Geodesic distance: 10.45 meters
39     - Reference path: 7 waypoints showing multi-level navigation
40
41   Key Differences from R2R
42
43   1. Much larger dataset: 146,304 episodes vs 10,819 in R2R
44   2. Smaller vocabulary: 2,504 words vs 2,711 in R2R
45   3. Longer instructions: Multi-sentence instructions with padding
46   4. Multi-level navigation: Instructions involving stairs and elevation
47     changes
48   5. Structured vocabulary: Separate stoi/itos mappings plus
49     UNK_INDEX/PAD_INDEX
50
51   Dataset Statistics
52
53   - Distance range: 3.46m - 30.71m (sampled from first 1,000 episodes)
54   - Path length range: 5-7 waypoints (more consistent than R2R)
55   - Scene coverage: Uses MP3D scenes like R2R
56
57   The EnvDrop dataset appears to be an augmented or expanded version of R2R
58   with more episodes,
59   multi-level navigation scenarios, and structured tokenization with explicit
60   padding and unknown
61   token handling.

```

scaleIn分析

```

代码块
1   Top-level Structure
2
3   - episodes: List of 155,098 navigation episodes (largest dataset so far)
4   - instruction_vocab: Empty vocabulary structure (all fields are empty/None)
5
6   Individual Episode Structure
7
8   Each episode contains the same basic structure as R2R and EnvDrop, but with
9   some key differences:
10
11   - episode_id: Unique identifier (integer, starts from 0)

```



```
11 - trajectory_id: Complex ID string with pattern  
scalevln_{scene_id}__fake_{start}_{end}  
12 - scene_id: HM3D scene identifier (format:  
"hm3d/{scene_id}/{scene_id}.basis.glb")  
13 - start_position: Starting coordinates [x, y, z] (list of floats)  
14 - start_rotation: Starting orientation [x, y, z, w] (list of floats)  
15 - goals: Target position with radius (list of dict with 'position' and  
'radius')  
16 - info: Always contains None for geodesic_distance  
17 - instruction: Navigation instruction containing:  
18 - instruction_text: Natural language instruction string (present)  
19 - instruction_tokens: Always None (not tokenized)  
20 - reference_path: Optimal path coordinates (list of [x, y, z] positions)
```

Key Differences from Other Datasets

1. Missing Data:
 - No geodesic_distance values (all None)
 - No tokenized instructions (all None)
 - Empty vocabulary structure
2. Scene Type: Uses HM3D (Habitat-Matterport 3D) scenes instead of MP3D
3. Trajectory ID Pattern: Complex naming with scalevln_ prefix and __fake_ suffix
4. Goal Radius: Consistently 0.25 meters (much smaller than R2R/EnvDrop's 3.0m)
5. Scale: 155,098 episodes (largest dataset examined)

Sample Data

```
- Instruction: "turn around and walk out of the bedroom . turn right and  
walk down the stairs . stop  
at the bottom of the stairs ."  
- Start position: [6.18, 0.07, 1.75]  
- Goal position: [-0.08, -2.53, 0.07] (significant elevation change)  
- Success radius: 0.25 meters (much more precise)  
- Reference path: 7 waypoints showing multi-level navigation
```

Dataset Characteristics

```
- Scene format: Uses .basis.glb files (compressed 3D format)  
- Navigation complexity: Multi-level navigation with stairs  
- Trajectory naming: Suggests synthetic or augmented data ("fake" in IDs)  
- Precision: Much smaller success radius indicates more precise navigation  
tasks
```

The ScaleVLN dataset appear

2. 评价采用 VLN 传统指标（NE/SR/OS/SPL），官方实验聚焦 unseen split。

轨迹数据（图像-动作对）

官方提供基于 R2R/RxR（MP3D 环境）收集的 observation-action 轨迹下载，解压到 `data/trajectory_data/`。EnvDrop 与 ScaleVLN 子集可按说明自行采集。

R2R

R2R 轨迹数据包含两个主要文件：

- `annotations.json` (5.5MB) - 轨迹标注数据
- `images.tar.gz` (7.9GB) - 对应的图像数据

annotations.json 文件结构

顶层结构

代码块

```
1  [
2    {
3      "id": 577,
4      "video": "images/17DRP5sb8fy_r2r_000577",
5      "instructions": ["指令1", "指令2", "指令3"],
6      "actions": [-1, 3, 3, 3, 1, 1, 2, ...]
7    },
8    ...
9  ]
```

字段详细说明

字段名	类型	含义
id	int	轨迹唯一标识符
video	str	对应图像数据的目录路径
instructions	list[str]	该轨迹对应的多个自然语言指令 (通常3个)
actions	list[int]	导航动作序列

数据统计信息

- 总轨迹数量: 3,603 个
- 平均指令数量: 3.0 个/轨迹
- 平均动作序列长度: 60.9 步

- **动作序列长度范围:** 24-196 步

动作编码系统

动作使用数字编码，包含以下4种类型：

含义	出现频率	作用推测
起始标记	1.60%	序列开始
前进/直行	63.10%	最主要的导航动作
左转	18.10%	方向调整
右转	17.10%	方向调整

轨迹长度分布

- **短轨迹** (≤ 40 步): 441 个 (12.2%)
- **中等轨迹** (41-80步): 2,649 个 (73.5%)
- **长轨迹** (> 80 步): 513 个 (14.2%)

图像数据结构

目录组织

代码块

```
1  images/
2  |— 17DRP5sb8fy_r2r_000577/
3  |   |— rgb/
4  |       |— 001.jpg
5  |       |— 002.jpg
6  |       |— ...
7  |— 1LXtFkjw3qL_r2r_001948/
8  |   |— rgb/
9  |       |— ...
10 |— ...
```

命名规则

- **目录名:** {场景ID}_r2r_{轨迹ID} (如 17DRP5sb8fy_r2r_000577)
- **图像文件:** rgb/{序号}.jpg (3位数命名)

轨迹示例

轨迹 ID 577:

- **视频路径:** images/17DRP5sb8fy_r2r_000577

- **样例指令:** "Walk past the dining table and take a left into the hallway. In the hallway walk into the bathroom and stop on the rug."
- **动作序列:** [-1, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 1, 1, 3, 1, 3, 1, 1, 1, 2, ...] (57步)
- **动作模式:** 包含多次连续转向(3)和前进(1)，模拟复杂的室内导航

动作模式分析

1. 连续性: 动作序列表现出明显的连续性模式

- 1->1 (连续前进): 最常见转换
- 3->3 (连续右转): 第二常见
- 2->2 (连续左转): 第三常见

2. 导航策略:

- **前进** 是主要动作 (63.1%)
- **转向动作** (左转+右转) 占 35.2%
- 不同长度轨迹的动作分布相对一致

数据集用途

这个轨迹数据集专门用于:

1. **视觉语言导航训练:** 提供观察-动作对序列
2. **多模态学习:** 结合自然语言指令和视觉观察
3. **序列建模:** 学习动作序列的时间依赖关系
4. **流式处理:** 支持在线决策和逐步导航

与 R2R 数据集的关系

- **轨迹数据** 是 **R2R VLN-CE 数据集** 的增强版本
- **VLN-CE** 提供离散的导航任务定义
- **轨迹数据** 提供连续的导航执行过程
- 两者结合可支持端到端的视觉语言导航系统训练

Trajectory Data 和 Datasets的关系

Datasets (`data/datasets/`)

这些数据集包含导航 episode，具体包括:

- **指令 (Instructions)** : 自然语言的导航命令
- **环境 (Environments)** : 3D 场景 (MP3D/HM3D)

- 目标（Goals）：目标位置
- 参考路径（Reference paths）：最优路线
- 示例：R2R、EnvDrop、ScaleVLN、RxR

Trajectory Data （data/trajectory_data/）

这里包含来自真实智能体导航的已收集观测-动作轨迹：

- 训练过程中收集的真实导航经验
- 智能体交互产生的观测-动作对
- Dagger（Dataset Aggregation）方式收集的数据
- 智能体实际走过的路径（不仅仅是参考路径）

关键差异（Key Differences）

维度	数据集（Datasets）	轨迹数据（Trajectory Data）
来源	人工标注	智能体导航
内容	指令 + 参考路径	观测 + 动作
目的	从零训练	微调 / 提升
格式	带元数据的 episode	轨迹序列
质量	黄金标准	真实智能体行为

训练流程中的关系（Relationship in Training Pipeline）

1. 第一阶段训练（Stage One Training）

- 使用数据集（R2R、EnvDrop 等）
- 基于人类示范训练智能体
- 学习基础导航技能

2. Dagger 收集（Dagger Collection）

- 智能体在环境中导航
- 从真实经验中收集 trajectory_data
- 存储观测-动作对

3. 第二阶段训练（Stage Two Training）

- 使用 trajectory_data 做微调
- 提升智能体性能
- 从真实导航经验中学习

实践示例（Practical Example）

- **数据集 Episode:**

指令: “去厨房并在冰箱旁停下”

参考路径: `[waypoint1, waypoint2, waypoint3]`

- **轨迹数据:**

观测: `[camera_image, position, heading]`

执行动作: `"TURN_LEFT"`

观测: `[new_camera_image, new_position, new_heading]`

执行动作: `"MOVE_FORWARD"`

.....

为何两者都需要 (Why Both Are Needed)

1. 数据集提供高质量的人类示范, 用于初始训练;
2. 轨迹数据提供真实的智能体经验, 用于改进;
3. 二者结合的训练方式比单独使用任一来源表现更佳。

目录模板 (复查用)

- README 给出了最终的 `data/` 树形结构 (`datasets/`、`scene_datasets/`、`trajectory_data/`、`dagger_data/`、`co-training_data/`)。 [GitHub](#)

联合训练的通用多模数据 (可选的第二阶段共训)

- LLaVA-Video-178K (视频 QA)、ScanQA/SQA3D (3D 场景问答)、MMC4-core (图文交错) 置于 `data/co-training_data/`, 具体子目录结构 README 已列出。 [GitHub](#)
- 论文给出的数据配方与规模 (用于再现论文二阶段共训比例):
 - VLA (导航专用) 样本: MP3D 60 场景采 45 万 clip (源自 R2R、EnvDrop、RxR); 再加 HM3D 700 场景采 30 万 clip (ScaleVLN 子集)。
 - DAgger 纠正示例: 24 万条, 作为混合训练的一部分。
 - 视频 VQA: 24.8 万 (LLaVA-Video-178K + ScanQA); 图文交错: 23 万 (MMC4)。
- 论文中的真实世界评测设置与端到端时延 (Unitree Go2 + D455, 相机上仰/上视; 推理 4 个动作约 0.27s, 通信 0.2s 室内/1.0s 室外): 用于你对实时性的期望值校准。

LLaVA-Video-178K (视频 QA) 分析(1.28TB)

下载

代码块

```
1 # 这里还缺少了申请的那一部分, 只是下载了json
```

```
2
3 hf download --repo-type dataset chchnii/StreamVLN-ScanQA-SQA3D-Data --local-dir ./ScanNet
```

LLaVA-Video-178K 是一个用于**视频指令微调 (video instruction tuning)** 的高质量**合成数据集**，覆盖三类任务：**详细字幕 (caption)**、**开放问答 (open_ended QA)**、**多选问答 (multi_choice QA)**

规模概览：自建部分含 **178,510** 条字幕、**960,792** 条开放问答、**196,198** 条多选问答；同时打包了 NeXT-QA/ActivityNetQA/PerceptionTest/LLaVA-Hound 等用于复现

字段含义 (Hugging Face Viewer 显示的列)

在任意子配置/切分里，典型样本包含以下字段（以 HF Viewer 展示为准）：`id`、`conversations`、`data_source`、`video`。

- **id : string**

样本唯一标识。例如 `028CE`、`14YU9` 等。

- **conversations : List[Turn]**

对话格式的**指令-回答**，长度通常为 2 (human→gpt)。

- `from` : 说话者，取值为 `"human"` 或 `"gpt"`。
- `value` : 文本内容。**视频占位**以 `"<image>\n"` 开头，后接指令或答案文本。
例：在 `caption` split 中，`human` 的 `value` 可能是 `"<image>\nCharacterize each scene..."`，`gpt` 给出**长段视频描述**；`open_ended` 为问答；`multi_choice` 的题干与选项通常写在 `human` 的文本里，`gpt` 给出**最终答案/解析**。[Hugging Face](#)

- **data_source : string**

来源域标记，用于区分视频来源（如学术集或 YouTube 等）。（数据仓库按 `*_academic_v0_1`、`*_youtube_v0_1` 等目录划分，`data_source` 与之对应。）
[Hugging Face](#)

- **video : string**

视频文件相对路径（位于对应子配置下的打包视频 `.tar.gz` 中）。HF Viewer 示例显示如 `academic_source/Charades/028CE.mp4`。[Hugging Face+1](#)

在 HF Viewer 里可直接看到：列名、示例 `conversations`（两条 turn）、`id`，以及 `video` 指向的相对路径。[Hugging Face](#)

三个任务的最小“样例”结构（示意）

注意：下列为**结构示意**，真实内容以仓库文件为准。

1. caption (视频描述)

`conversations[0].value` : `"<image>\nCharacterize each scene ..."` (指令)

id string · lengths	conversations list · lengths	data_source string · classes	video string · lengths
5e9 30.7%	2 100%	0_30_s_aca... 100%	34e37 30.7%
028CE	[{ "from": "human", "value": " <image>\nAnalyze the video, focusing on its cinematography and narrative structure." }, { "from": "gpt", "value": "The video begins with a view of a garage or basement, characterized by a white door in the background. On the floor, there is a vacuum cleaner and a small box with a decorative pattern. A person dressed in black clothing and black shoes enters the frame and approaches the vacuum cleaner. The person squats down and starts to interact with the vacuum cleaner, adjusting or inspecting	0_30_s_academic_v0_1	academic_source/Charades/028CE.mp4

2. open_ended (开放问答)

`conversations[0].value`: "`<image>\nQuestion: ...`" (可能询问动作、因果、时序等)

`conversations[1].value`: **自然语言答案** (一句到一段)。

3. multi_choice (多选问答)

`conversations[0].value`: "`<image>\nQ: ... \nOptions: (A) ... (B) ... (C) ... (D) ...`"

`conversations[1].value`: **标准答案** (常见形式为选项或含解析的文本)。(具体格式见对应 `mcprocessed.json`，存储为 LFS，需下载查看。)

MMC4-core (图文交错) 分析

代码块

```
1 # 感觉应该是mmc4-ff里面的，也就是那个218G的
2 hf download --repo-type dataset chchnii/jmhessel/mmc4-core-ff --local-dir
  ./MMC4-core
```


MMC4 (Multimodal C4) 是在 C4 文本语料基础上，把网页里的图片与句子按语义对齐后“交错 (interleaved)”存放的超大规模图文语料，面向需要“图文交错输入”的多模态预训练/指令学习框架（如 Flamingo 风格的 in-context 多图文推理、检索增强、多图对比等）。论文给出构建方法与质量评估（CLIP 线性分配对齐、88% 话题相关等）主要用作图文对齐的

每条样本的字段说明 (schema)

Hugging Face 预览里直接列出了列名与示例，我们结合官方 README 的定义逐一解释：

1. url : string

- 该文档（网页）的原始 URL。

2. text_list : sequence (list of strings)

- 将网页正文切分为句子的列表（按句存放，顺序即原文顺序）。后续的图像对齐与相似度都是在 $image \times sentence$ 粒度上进行。

3. image_info : list (每张图像一个对象)，对象内常见键：

- image_name : string

下载后建议保存的文件名（通常是内容哈希+后缀）。

- raw_url : string

图片原始下载地址（设计上不再二次分发原图，尊重网页方删除；若 URL 失效，可按 README 的脚本尝试回落）。

- matched_text_index : int

该图片最匹配的句子在 text_list 中的索引（基于 CLIP ViT-L/14 的线性分配算法得到的对齐结果）。

- matched_sim : float

上述图片与匹配句子之间的 CLIP 相似度（ViT-L/14）。

- face_detections : None / list

人脸检测结果；在 fewer-faces 版本里通常为 None（即过滤后无脸）。

4. similarity_matrix : sequence (二维矩阵)

- 形状为 $[\#images \times \#sentences]$ 的 CLIP 相似度矩阵，`similarity_matrix[i, j]` 表示第 i 张图与第 j 个句子的相似度；`matched_text_index / matched_sim` 就来自这张矩阵的最佳分配。

5. could_have_url_duplicate : int64 (0/1)

- 标注该 URL 可能在 Common Crawl 的不同时点被重复抓取。官方做了下采样，期望每 URL 只出现一次，但仍可能重复；若要更严格去重，可过滤值为 1 的样本。

代码块

```
1 sample = {
2     "url": "https://example.com/how-to-brew-coffee",
```

```

3     "text_list": [
4         "手冲咖啡需要新鲜的中度烘焙豆。",
5         "将水加热到92-95°C并缓慢注水。"
6     ],
7     "image_info": [
8         {
9             "image_name": "img_0001.jpg",
10            "raw_url": "https://img.example.com/beans.jpg",
11            "matched_text_index": 0,
12            "matched_sim": 0.41,
13            "face_detections": None
14        },
15        {
16            "image_name": "img_0002.jpg",
17            "raw_url": "https://img.example.com/kettle.jpg",
18            "matched_text_index": 1,
19            "matched_sim": 0.57,
20            "face_detections": None
21        },
22        {
23            "image_name": "img_0003.jpg",
24            "raw_url": "https://img.example.com/pour.jpg",
25            "matched_text_index": 1,
26            "matched_sim": 0.49,
27            "face_detections": None
28        }
29    ],
30    "similarity_matrix": [ # 形状: 3张图 × 2句
31        [0.41, 0.12], # img_0001 对 句0/句1 的 CLIP 相似度
32        [0.05, 0.57], # img_0002
33        [0.09, 0.49] # img_0003
34    ],
35    "could_have_url_duplicate": 0
36 }

```

ScanQA/SQA3D（3D 场景问答）分析

目录结构

代码块

```

1  data/co-training_data/ScanNet/
2  |—— .cache/huggingface/           # HuggingFace缓存目录
3  |—— scanqa_annotations.json        # ScanQA标注文件（10.8MB）
4  |—— sqa3d_annotations.json         # SQA3D标注文件（12.8MB）
5  |—— .gitattributes                 # Git属性文件

```

数据集概览

ScanQA 数据集

- 样本数量: 26,515 个问答对
- 场景覆盖: 562 个不同的ScanNet场景
- 问题复杂度: 平均 18.7 个单词/问题
- 数据大小: 10.8MB (JSON格式)

SQA3D 数据集

- 样本数量: 26,623 个问答对
- 场景覆盖: 518 个不同的ScanNet场景
- 问题复杂度: 平均 36.2 个单词/问题 (更复杂)
- 数据大小: 12.8MB (JSON格式)

1. 数据格式

统一的JSON结构

代码块

```
1  {
2      "id": 0,
3      "video": "posed_images/scene0000_00",
4      "conversations": [
5          {
6              "from": "human",
7              "value": "<video>\nWhat is in the right corner of room by
8              curtains? Answer the question using one word or one phrase."
9          },
10         {
11             "from": "gpt",
12             "value": "Brown cabinet with tv sitting in it."
13         }
14     ]
15 }
```

关键字段说明

- **id**: 唯一标识符
- **video**: 视频路径引用, 指向 `posed_images/场景ID`
- **conversations**: 问答对话列表

- `from`: "human" (问题) 或 "gpt" (答案)
- `value`: 具体的对话内容

2. 问题类型差异

ScanQA 特点

- **简单直接**: "What is in the right corner of room by curtains?"
- **视觉定位**: 主要关注物体识别和空间关系
- **简短回答**: 要求"one word or one phrase"

SQA3D 特点

- **复杂上下文**: "I am facing a window and there is a desk on my right and a chair behind me. What color is the desk to my right?"
- **空间推理**: 需要理解观察者视角和空间配置
- **长问题**: 平均长度是ScanQA的2倍

dagger_data分析

训练与推理细节 (Train & Inference)

环境与依赖 (作者测试配置)

- Python 3.9、PyTorch 2.1.2、CUDA 12.4。 [GitHub](#)
- Habitat 安装建议: `habitat-sim==0.2.4 withbullet headless` ; `habitat-lab` 与 `habitat-baselines` 以 `-e` 安装。 [GitHub](#)
- 克隆仓库后进入根目录: `git clone` <https://github.com/OpenRobotLab/StreamVLN.git> && `cd StreamVLN` (仓库名在 README 链接中也指向该组织) 。 [GitHub](#)
- 资源: https://internrobotics.github.io/user_guide/internnav/quick_start/installation.html

Model-Specific Requirements

Models	Minimum GPU Requirement		System RAM (Train/Inference)
	Training	Inference	
StreamVLN & InternVLA-N1	A100	RTX 4090 / A100	80GB / 24GB
NavDP (VN Models)	RTX 4090 / A100	RTX 3060 / A100	16GB / 2GB
CMA (VLN-PE Small Models)	RTX 4090 / A100	RTX 3060 / A100	8GB / 1GB

- Lora: 4张32G
- 全量4张A800 需要一天的时间

推理流程

低配版资源运行

通过量化和调整数据格式的方式，在16G v100上面可运行：https://gitcode.com/kason-huang/vln_platform

主要修改：
<https://gitcode.com/Winsleo/StreamVLN/commit/29d0c32ee6463b2054dfc0edd41e551c4e8b848c?ref=poor> 这里还不包含量化，还需要看vln_platform里面的

用24G 4090也可以，不需要用量化

docker里面是可以运行的

代码块

```
1 export DATA_PATH=/mnt/sfs_turbo/data-platform/
2 docker compose -f .docker/docker-compose.yml -f .docker/docker-
  compose.streamvln.yml up -d
3 docker compose -f .docker/docker-compose.yml -f .docker/docker-
  compose.streamvln.yml stop
4 docker exec -it streamvln bash
5 bash scripts/vln_eval_multi_gpu.sh
6
```

R2R-Val_unseen-运行结果

评估概览

- 运行命令：

```
代码块
1 export DATA_PATH=/mnt/sfs_turbo/data-platform/
2 docker compose -f .docker/docker-compose.yml -f .docker/docker-
  compose.streamvln.yml up -d
3 docker compose -f .docker/docker-compose.yml -f .docker/docker-
  compose.streamvln.yml stop
4 docker exec -it streamvln bash
5 bash scripts/vln_eval_multi_gpu.sh
```

- 数据集: R2R Room-to-Room
- 验证集: val_unseen
- 模型: StreamVLN
- 总评估片段: 1,839 个
- 评估时间: 2025年9月30日 - 10月1日

核心性能指标

指标	数值	说明
成功率 (Success Rate)	53.45%	成功到达目标位置的比例
SPL (Success weighted by Path Length)	0.4857	考虑路径效率的成功率
OS (Oracle Success)	0.5987	预测路径与最短路径的重叠度
平均导航误差 (Navigation Error)	5.36米	距离目标位置的平均误差
平均碰撞次数	10.28次	每个片段的平均碰撞数
平均步数	73.19步	每个片段的平均行动步数

按场景分析

表现最佳的场景 (成功率>60%):

- X7HyMhZNoso: 65.25% 成功率，平均导航误差仅3.43米
- 8194nk5LbLH: 61.54% 成功率
- TbHJrupSAjP: 57.95% 成功率，平均导航误差4.72米

表现较弱的场景 (成功率<50%):

- oLBMNvg9in8: 42.37% 成功率，碰撞次数高达21.56次
- Z6MFQCViBuw: 49.06% 成功率，平均导航误差9.62米

关键发现

1. 导航能力: 模型在约一半的场景中能够成功导航到目标位置，显示出基本的语言理解和空间导航能力。

- 2. **路径效率:** SPL (0.4857) 低于成功率 (0.5345)，表明模型在找到最优路径方面仍有改进空间。
- 3. **碰撞问题:** 平均每片段10.28次碰撞表明模型在障碍物避让方面需要优化。
- 4. **场景依赖性:** 不同场景的性能差异显著，说明模型在某些环境布局中表现更好。

可视化输出

- 生成了1,841个片段的可视化数据
- 每个片段包含:
 - `actions.json` : 模型预测的动作序列
 - `actions_human.json` : 人类参考动作序列
 - `frames/` : 导航过程中的第一人称视角图像帧

建议

- 1. **减少碰撞:** 改进障碍物检测和避让策略
- 2. **路径优化:** 提升路径规划效率以改善SPL指标
- 3. **场景泛化:** 针对表现较弱的环境场景进行专项优化
- 4. **长距离导航:** 重点解决导航误差随距离增加的问题

训练阶段划分（论文 & 脚本）

概览

代码块

```
1  训练流程概述
2
3  StreamVLN采用三阶段训练策略：
4  1. Stage-one：预训练阶段（有监督训练）
5  2. Dagger Collection：数据收集阶段
6  3. Stage-two：联合训练阶段（有监督训练）
7
8  第一阶段：预训练（Stage-one）
9  只用 oracle 轨迹微调 1 个 epoch
10 使用数据集（132G）
11  - R2R（Room-to-Room）：data/trajectory_data/R2R
12  - RxR（Room-across-Room）：data/trajectory_data/RxR
13  - EnvDrop：data/trajectory_data/EnvDrop
14  数据特点：预收集的观察-动作对序列
15  训练配置：
16  - group_by_task=False
17  - multi_task_training=False（未启用）
```

18 - 单任务训练, 专注于 VLN 任务

19

20 关键配置

- 21 - 基础模型: `lmms-lab/LLaVA-Video-7B-Qwen2`
- 22 - 视觉编码器: `google/siglip-so400m-patch14-384`
- 23 - 训练轮数: 1 epoch
- 24 - 序列长度: 32768 tokens
- 25 - 视频帧数: 32 frames
- 26 - 历史步数: 8 steps
- 27 - 未来步数: 4 steps

28

29 训练参数

- 30 `learning_rate: 2e-5`
- 31 `mm_vision_tower_lr: 5e-6`
- 32 `per_device_train_batch_size: 2`
- 33 `gradient_accumulation_steps: 2`
- 34 `warmup_ratio: 0.075`
- 35 `lr_scheduler_type: cosine_with_min_lr`

36

37 命令: Stage-1: `sbatch scripts/streamvln_train_slurm.sh`

38

39 第二阶段: Dagger数据收集

40 用 stage-1 模型在环境中 rollout, 利用 Habitat 的最短路专家做纠偏, 收集 24 万样本

41 数据集配置

- 42 - 主要数据集: R2R, RxR, EnvDrop
- 43 - 数据路径: `data/datasets/r2r/train/train.json.gz`
- 44 - 标注路径: `data/trajectory_data/R2R_V1-3/annotations.json`

45

46 Dagger参数

- 47 - 更新大小: 160,000 samples
- 48 - 提交频率: 每50次更新提交一次
- 49 - 模型推理概率: 0 (纯专家数据收集)
- 50 - 数据迭代: 3次

51

52 环境配置

- 53 - 最大步数: 500 steps
- 54 - 成功距离: 3.0 meters
- 55 - 传感器配置: RGB(640x480) + Depth(640x480)
- 56 - 步长: 0.25 meters
- 57 - 转角: 15 degrees

58

59 命令: DAgger 采集: `sh scripts/streamvln_dagger_collect.sh`

60

61 第三阶段: 联合训练 (Stage-two)

62 在 VLA + DAgger + 通用多模数据 (视频 QA、MMC4 图文) 的混合上继续训练 1 个 epoch

63 多任务数据集

64 1. VLN导航数据 (???)

65 - 预训练轨迹数据: *R2R*, *RxR*, *EnvDrop*
66 - *Dagger*收集数据: *data/dagger_data/{R2R,RxR,EnvDrop}*
67
68 2. 视频问答数据 (LLaVA-Video-178K) (1.28TB)
69 - 学术视频: 0-30秒和30-60秒视频问答数据
70 - YouTube视频: 随机采样140,000个样本
71 - 数据子集: *NextQA*, *ActivityNet*, *Ego4D*, *Charades*
72
73 3. 3D场景问答 (*ScanQA*) (需要发送申请)
74 - *ScanNet*标注: *scanqa_annotations.json*
75 - 3D问答标注: *sqa3d_annotations.json*
76
77 4. 大规模图文对齐 (*MMC4*) (20G)
78 - 核心数据: *MMC4-core/*
79 - 采样策略: 最后1400个样本
80
81 联合训练配置
82 - 任务分组: 启用 *group_by_task=True*
83 - 多任务训练: 启用 *multi_task_training=True*
84 - 任务ID分配:
85 - 导航任务: 0
86 - 视频问答: 1
87 - 3D问答: 2
88 - 图文对齐: 3
89
90 训练超参数
91 *learning_rate: 2e-5*
92 *mm_vision_tower_lr: 5e-6*
93 *warmup_ratio: 0.03*
94 *lr_scheduler_type: cosine*
95 *num_train_epochs: 1*
96 *model_max_length: 32768*
97
98 模型架构配置
99 可训练模块
100 - *mm_tunable_parts="mm_vision_tower,mm_mlp_adapter,mm_language_model"*
101 - 视觉编码器、多模态投影器、语言模型同时训练
102
103 视觉处理
104 - 图像分辨率: 任意分辨率, 最大9宫格 (1x1) 到 (6x6)
105 - 视频采样: 均匀采样32帧
106 - 数据增强: 颜色抖动、随机后处理、锐化调整
107
108 上下文管理
109 - 历史窗口: 8步对话历史
110 - 未来预测: 4步动作预测
111 - 记忆机制: *Token pruning for long sequences*

```
112
113 分布式训练配置
114 硬件配置
115 - 节点数: 4 nodes
116 - GPU数: 8 GPUs per node
117 - 总批大小: 64 (4 nodes × 8 GPUs × 2 batch size)
118
119 优化设置
120 - 混合精度: BF16训练
121 - 梯度检查点: 启用以节省内存
122 - DeepSpeed: Zero2优化
123 - Torch编译: 启用 torch_compile=True
124
125 命令: Stage-2: sbatch scripts/streamvln_stage_two_train_slurm.sh
126
127 这个三阶段训练策略使得StreamVLN能够从预训练的轨迹数据中学习基础导航能力，通过Dagger收集
    真实环境数据，最后在多任务设置下联合优化，实现强大的视觉语言导航能力。
128
129 Key Differences Between Stages
130 1. Stage One: Only uses pre-collected trajectory data (R2R, RxR, EnvDrop)
131 2. Stage Two: Combines trajectory data + dagger collected data + multi-modal
    co-training data
132 3. Multi-task Training: Only Stage Two enables --group_by_task True and --
    multi_task_training True
133 4. Learning Rate: Stage One uses warmup_ratio 0.075, Stage Two uses 0.03
134 5. Data Sources: Stage Two adds MMC4, ScanQA, and LLaVA-Video-178K datasets.
135 This three-stage approach allows the model to first learn basic navigation
    skills, then collect real-world interaction data, and finally fine-tune with
    multi-modal tasks for robust performance.
136
137 阶段一:
138 数据纯净，专注于 VLN 任务
139 学习基础的视频-语言-动作映射
140 阶段二:
141 数据多样性最高
142 多任务学习增强泛化能力
143 混合原始轨迹 + Dagger 收集轨迹
144
```

AutoDL

第一阶段（有监督训练）

资源诉求：

Lora: 4张32G

全量4张A800

代码块

1

autodl环境准备

1. 1张 A800进行环境搭建

a. 注意系统盘与数据盘的大小



2. 配置git ssh、obsutil

3. 下载代码 git clone [git@github.com:kason-huang/StreamVLN.git](https://github.com/kason-huang/StreamVLN.git)

a. 速度很慢（配置加速，官网有说明）多换几次机器，有的机器快的，有的机器就是慢

b. 这挺快



4. 搭建运行环境 <https://github.com/kason-huang/StreamVLN/tree/hqk-dev/scripts>

a. 可以使用conda

代码块

```
1 # 初始化（会往 ~/.bashrc 写入 hook）
2 /opt/conda/bin/conda init bash || ~/miniconda3/bin/conda init bash
3
4 # 重新进入一个登录 bash（让修改立刻生效）
5 exec bash -l
6
7 # 然后激活
8 conda activate streamvln
```

a. <https://github.com/kason-huang/StreamVLN/tree/hqk-dev>

5. 打一个镜像

6. 下载checkpoint 直接运行utils/model_download 即可

a. opt目录，可以登录

7. 上传数据（先试试obsutil，不行就xshell）

代码块

```
1  /obsutil config -i=HPUAXIGJCSQZZAWTAFGF -  
   k=v1756rW0y50JDF5rAfmej9uhLHOKtCJotvTGA6cZ -e=https://obs.cn-east-  
   3.myhuaweicloud.com  
2  
3  # 一个个拷贝吧  
4  obsutil cp obs://data-platform-shanghai/data/streamvln_data streamvln_data -r -  
   f  
5  obsutil cp obs://data-platform-shanghai/data/streamvln_data/datasets datasets -  
   r -f  
6  obsutil cp obs://data-platform-shanghai/data/streamvln_data/scene_datasets  
   scene_datasets -r -f  
7  obsutil cp obs://data-platform-shanghai/data/streamvln_data/trajectory_data  
   trajectory_data -r -f
```

streamvln_train.py

代码块

```
1  ● First Stage Training Code Logic Summary  
2  
3  Core Training Pipeline  
4  
5  1. Single-Task Focus:  
6  nav_dataset = VLNAActionDataset(tokenizer=tokenizer, data_args=data_args,  
   task_id=0)  
7  dataset = [nav_dataset]  
8  - Only navigation data (task_id=0) in stage one  
9  - No co-training datasets (QA, ScanQA, MMC4 disabled)  
10  
11  2. Transfer Learning Architecture:  
12  model = StreamVLNForCausalLM.from_pretrained(model_args.model_name_or_path)  
13  - Pre-trained LLaVA-Video-7B-Qwen2 as foundation  
14  - Selective fine-tuning of specific components  
15  - Preserves existing multi-modal capabilities  
16
```

```

17 3. Selective Parameter Updates:
18 tunable_parts = model_args.mm_tunable_parts.split(",")
19 if "mm_vision_tower" in tunable_parts:
20     for name, param in model.named_parameters():
21         if "vision_tower" in name:
22             param.requires_grad_(True)
23 if "mm_mlp_adapter" in tunable_parts:
24     for p in model.get_model().mm_projector.parameters():
25         p.requires_grad = True
26 if "mm_language_model" in tunable_parts:
27     for name, param in model.named_parameters():
28         if "vision_tower" not in name and "mm_projector" not in name and
"vision_resampler"
29     not in name:
30         param.requires_grad_(True)
31
32 Temporal Context Integration
33 1. Sequential Modeling:
34 if data_args.num_history:
35     overwrite_config["num_history"] = data_args.num_history
36 if data_args.num_future_steps:
37     overwrite_config["num_future_steps"] = data_args.num_future_steps
38 - 8 history steps provide context
39 - 4 future steps enable action prediction
40 - 32 frames capture visual observations
41
42 2. Frame Processing Strategy:
43 sampled_indices = np.linspace(0, total_frames - 1, num_frames_to_sample,
dtype=int)
44 - Uniform temporal sampling from video sequences
45 - Multi-source support (frame directories, video files, ScanNet)
46 - Time instruction injection for temporal awareness
47
48 Data Flow Architecture
49
50 1. Video → Language → Action Pipeline:
51 Video Frames (32) → SigLIP Encoder → MLP Projection → Qwen2 Language Model
→ Action Tokens
52     ↓             ↓             ↓             ↓
53 Temporal Context  Visual Features  Multi-modal Fusion  Sequential Decision
54
55 2. Navigation-Specific Processing:
56 return data_dict["input_ids"], \
57         data_dict["labels"], \
58         data_dict["image"][0][0], \
59         None, self.task
60 - Input IDs: Language instructions + conversation history

```

```

61 - Labels: Target navigation actions
62 - Images: Processed video frames
63 - Task ID: Navigation task identifier
64
65 Optimization Strategy
66
67 1. Memory-Efficient Training:
68 if training_args.gradient_checkpointing:
69     model.enable_input_require_grads()
70 - Gradient checkpointing for large sequences
71 - Mixed precision (BF16) support
72 - DeepSpeed ZeRO-2 for distributed training
73
74 2. Learning Rate Hierarchy:
75 model.config.mm_vision_tower_lr = training_args.mm_vision_tower_lr
76 model.config.mm_projector_lr = training_args.mm_projector_lr
77 - Vision tower: Lower LR (preserve features)
78 - MLP adapter: Medium LR (learn projection)
79 - Language model: Higher LR (adapt to task)
80
81 Stage One Objectives
82
83 Primary Goal: Adapt pre-trained multi-modal model to VLN tasks
84 - Learn observation-to-action mapping
85 - Establish temporal consistency
86 - Preserve general multi-modal capabilities
87
88 Output: VLN-specialized checkpoint ready for:
89 - Second-stage co-training
90 - Real-world deployment
91 - Interactive navigation scenarios
92
93 This architecture enables efficient transfer learning while maintaining the
flexibility
94 needed for complex vision-and-language navigation tasks.

```

第三阶段：联合训练 (Stage-two) (有监督训练)

Vision-Language Action Data. We collect navigation-specific training data using the Habitat simulator across multiple public VLN datasets. Specifically, we collect 450K samples (video clips) from 60 Matterport3D [25] (MP3D) environments, sourced from R2R [7], R2R-EnvDrop [26] and RxR [8]. To further improve generalization through increased scene diversity, we incorporate an additional 300K samples from a subset of ScaleVLN [19], spanning 700 Habitat Matterport3D [27] (HM3D) scenes. In addition, we adopt the DAgger [28] algorithm to enhance the model’s robustness and generalization ability in novel scenes and during error recovery. Using Habitat’s shortest-path follower as the expert policy, we collect corrective demonstrations on model rollouts after the initial training stage. These DAgger-collected samples (240K) are then incorporated into the training set for co-training.

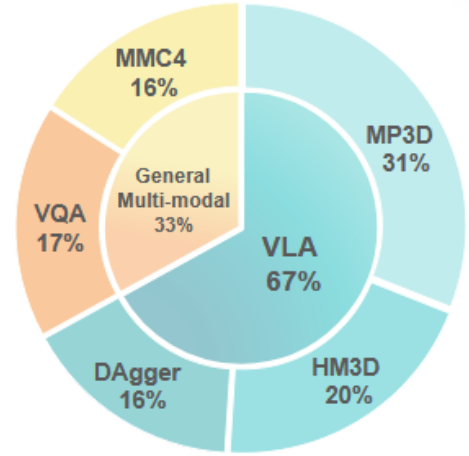


Figure 2: Co-Training Data Recipe of StreamVLN

论文实验

通过上面描述的三阶段的训练了模型，与其他模型的Performance对比如下

Method	Observation Encoder				R2R Val-Unseen				RxR Val-Unseen			
	Pano.	Odo.	Depth	S.RGB	NE↓	OS↑	SR↑	SPL↑	NE↓	SR↑	SPL↑	nDTW↑
HPN+DN* [33]	✓	✓	✓		6.31	40.0	36.0	34.0	-	-	-	-
CMA* [34]	✓	✓	✓		6.20	52.0	41.0	36.0	8.76	26.5	22.1	47.0
VLN⊙BERT* [34]	✓	✓	✓		5.74	53.0	44.0	39.0	8.98	27.0	22.6	46.7
Sim2Sim* [35]	✓	✓	✓		6.07	52.0	43.0	36.0	-	-	-	-
GridMM* [36]	✓	✓	✓		5.11	61.0	49.0	41.0	-	-	-	-
ETPNav* [16]	✓	✓	✓		4.71	65.0	57.0	49.0	5.64	54.7	44.8	61.9
ScaleVLN* [19]	✓	✓	✓		4.80	-	55.0	51.0	-	-	-	-
InstructNav [23]	-	-	-	-	6.89	-	31.0	24.0	-	-	-	-
AG-CMTP [37]	✓	✓	✓		7.90	39.2	23.1	19.1	-	-	-	-
R2R-CMTP [37]	✓	✓	✓		7.90	38.0	26.4	22.7	-	-	-	-
LAW [13]		✓	✓	✓	6.83	44.0	35.0	31.0	10.90	8.0	8.0	38.0
CM2 [14]		✓	✓	✓	7.02	41.5	34.3	27.6	-	-	-	-
WS-MGMap [15]		✓	✓	✓	6.28	47.6	38.9	34.3	-	-	-	-
ETPNav + FF [38]		✓	✓	✓	5.95	55.8	44.9	30.4	8.79	25.5	18.1	-
Seq2Seq [39]			✓	✓	7.77	37.0	25.0	22.0	12.10	13.9	11.9	30.8
CMA [39]			✓	✓	7.37	40.0	32.0	30.0	-	-	-	-
NaVid [3]				✓	5.47	49.1	37.4	35.9	-	-	-	-
MapNav [6]				✓	4.93	53.0	39.7	37.2	-	-	-	-
NaVILA [5]				✓	5.37	57.6	49.7	45.5	-	-	-	-
StreamVLN				✓	5.43	62.5	52.8	47.2	6.72	48.6	42.5	60.2
NaVILA† [5]				✓	5.22	62.5	54.0	49.0	6.77	49.3	44.0	58.8
UniNaVid† [4]				✓	5.58	53.3	47.0	42.7	6.24	48.7	40.9	-
StreamVLN†				✓	4.98	64.2	56.9	51.9	6.22	52.9	46.0	61.9

Table 1: Comparison with state-of-the-art methods on VLN-CE R2R and RxR Val-Unseen split. * indicates methods using the waypoint predictor from [34]. † denotes methods using additional training data beyond the R2R-CE and RxR-CE benchmarks.

Method	ScanQA				
	Bleu-4 ↑	Rouge ↑	Meteor ↑	Cider ↑	EM ↑
ScanRefer [40]	7.9	30.0	55.4	11.5	18.6
ScanQA [30]	10.1	33.3	64.9	13.1	21.0
3D-VisTA [41]	10.4	35.7	69.6	13.9	22.4
3D-LLM* [42]	12.0	35.7	69.4	14.5	20.5
LEO [43]	13.2	49.2	101.4	20.0	24.5
ChatScene* [44]	14.0	-	87.6	-	-
Scene-LLM* [45]	12.0	40.0	80.0	16.6	27.2
NaviLLM [46]	12.0	38.4	75.9	15.4	23.0
NaVILA (16 frames) [5]	15.2	48.3	99.8	19.6	27.4
StreamVLN (16 frames)	15.7	48.3	100.2	19.8	28.8

Table 2: Comparison on ScanQA [30] Val set. * indicates 3D LLMs with task-specific fine-tuning.

1. 围绕第三阶段的cotrain做了消融对比实验

R2R	RxR	Dagger	VL Data	ScaleVLN(150K)	NE↓	OS↑	SR↑	SPL↑
✓	✓				6.05	53.8	45.5	41.6
✓	✓	✓	VideoQA		5.47	57.8	50.8	45.7
✓	✓	✓	VideoQA+MMC4		5.43	62.5	52.8	47.2
✓	✓	✓	VideoQA+MMC4	✓	5.10	64.0	55.7	50.9
✓	✓		VideoQA+MMC4	✓	5.73	56.4	50.2	47.1
✓		✓	VideoQA+MMC4	✓	5.90	55.9	47.9	43.6

Table 3: Ablation study of different training data compositions on VLN-CE R2R Val-Unseen split.

- The first row shows the first-stage performance when training with only oracle navigation data. After collecting DAgger data, we co-train oracle data, DAgger data, and vision-language (VL) data
- For a fair comparison, the total number of VL Data is kept the same
- The results show that DAgger data plays a crucial role in boosting performance (+5.5 SR / +3.8 SPL)

新实验

项目分析

关键类分析

代码块

1 StreamVLN项目关键类与关系分析

- 2
- 3 1. 核心模型架构类
- 4 1.1 StreamVLNModel (stream_video_vln.py:14-31)
- 5 - 继承关系: LlavaQwenModel
- 6 - 功能: VLN任务的基础模型, 继承LLaVA-Video的多模态能力
- 7 - 关键特性:
- 8 - 支持历史观察记忆 (num_history)
- 9 - 冻结视觉投影适配器 (freeze_mm_mlp_adapter)
- 10 - 配置视觉塔参数
- 11
- 12 1.2 StreamVLNForCausalLM (stream_video_vln.py:32-479)
- 13 - 继承关系: Qwen2ForCausalLM + LlavaMetaForCausalLM
- 14 - 功能: 完整的VLN因果语言模型, 支持生成导航动作
- 15 - 核心方法:
- 16 - encode_rgb_d(): 编码RGB-D图像和位姿信息
- 17 - prepare_inputs_labels_for_multimodal(): 准备多模态输入
- 18 - forward(): 前向传播
- 19 - generate(): 生成导航动作序列
- 20
- 21 1.3 关系
- 22 StreamVLNModel (基础模型层)
- 23 - 职责: 提供多模态基础架构, 专注多模态基础能力
- 24 - 特点:
- 25 - 继承自LlavaQwenModel
- 26 - 配置视觉塔参数
- 27 - 管理历史观察记忆(num_history)
- 28 - 冻结视觉投影适配器
- 29
- 30 StreamVLNForCausalLM (完整模型层)
- 31 - 职责: 提供完整的VLN因果语言模型功能, 专注导航任务逻辑
- 32 - 特点:
- 33 - 继承语言模型能力(Qwen2ForCausalLM)
- 34 - 继承多模态元接口(LlavaMetaForCausalLM)
- 35 - 包含StreamVLNModel实例作为self.model
- 36 - 实现导航特定的编码和生成方法
- 37
- 38
- 39 数据流向: 输入数据 → StreamVLNForCausalLM → 委托给StreamVLNModel → 视觉编码 →
- 40 返回到StreamVLNForCausalLM → 多模态融合 → 输出/生成
- 41
- 42 2. 数据处理类
- 43 2.1 VLNActionDataset (vln_action_dataset.py:607-825)
- 44 - 继承关系: Dataset
- 45 - 功能: VLN行动数据集, 处理导航轨迹数据

- 关键特性：
 - 视频帧采样和预处理
 - 历史观察记忆构建
 - 对话式指令-动作对生成
 - 支持4种导航动作：停止、前进、左转、右转

2.2 LazySupervisedDataset (streamvln_train.py:933-1297)

- 继承关系: Dataset
- 功能: 通用监督数据集, 支持多任务训练
- 支持数据类型: 图像、视频、问答、对话

2.3 CombineDataset (streamvln_train.py:902-931)

- 继承关系: Dataset
- 功能: 组合多个数据集, 实现多任务训练
- 任务ID映射:
 - 导航任务: 0
 - 问答任务: 1
 - ScanQA: 2
 - 视频字幕: 3

3. 训练基础设施类

3.1 LLaVATrainer (llava/train/llava_trainer.py)

- 继承关系: transformers.Trainer
- 功能: 专门为LLaVA设计的训练器
- 特性: 支持多模态训练、混合精度、分布式训练

3.2 DataCollatorForSupervisedDataset (streamvln_train.py:1300-1433)

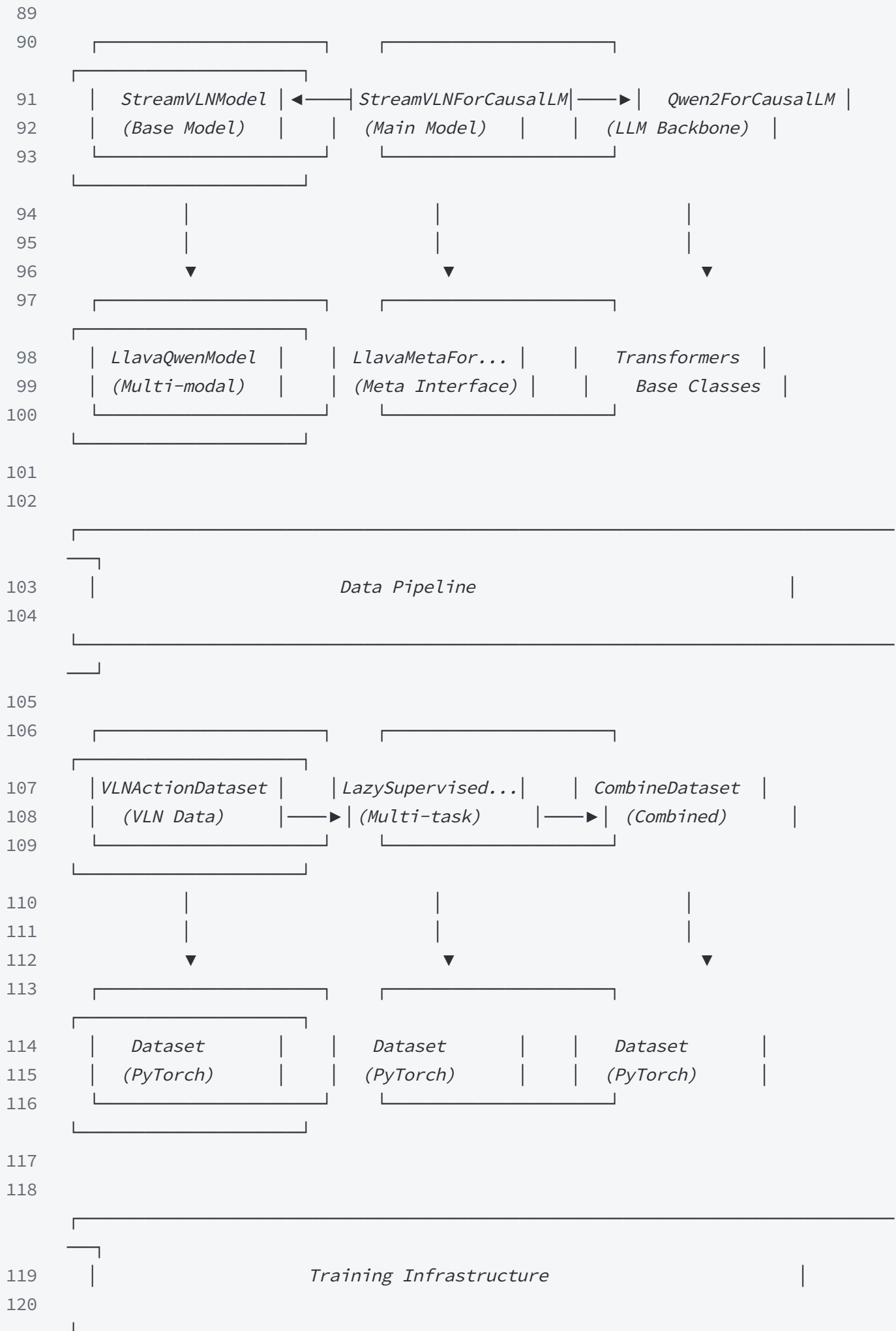
- 功能: 监督数据的数据整理器
- 处理数据: 输入ID、标签、图像、深度、位姿、时间ID等

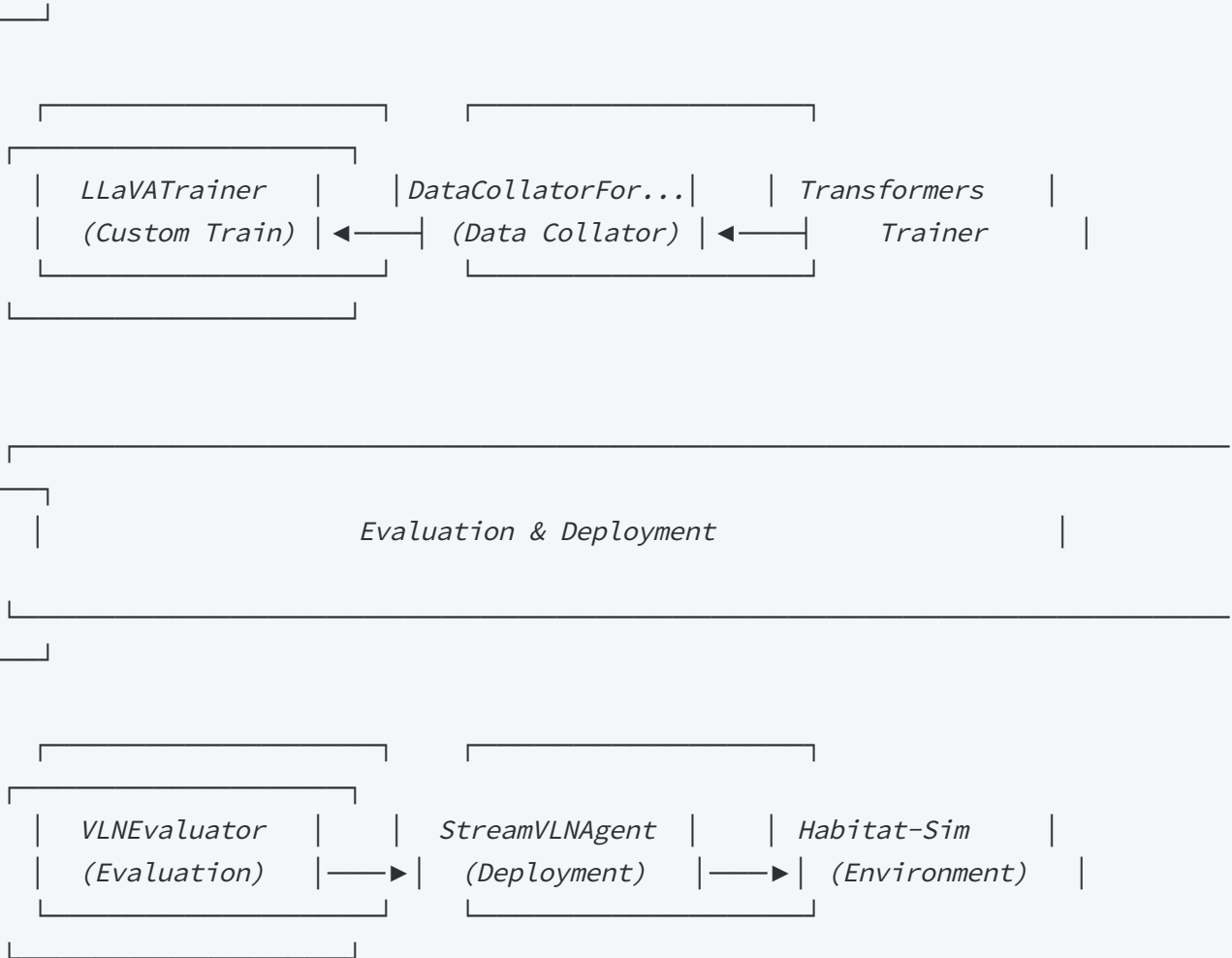
4. 评估和部署类

4.1 VLNEvaluator (streamvln_agent.py:31-100)

- 功能: VLN评估器, 处理真实环境中的导航任务
- 关键组件:
 - 多模态输入处理
 - 动作序列生成
 - 记忆管理
 - 环境交互

5. 类关系图





6. 关键数据流

6.1 训练数据流

原始数据 → VLNActionDataset/LazySupervisedDataset → CombineDataset → DataCollatorForSupervisedDataset → LLaVATrainer → StreamVLNForCausalLM

6.2 推理数据流

环境输入 → VLNEvaluator → StreamVLNForCausalLM → 动作生成 → 环境执行

6.3 多模态处理流

RGB图像 + 深度 + 位姿 → encode_rgbd() → 视觉特征 → 记忆特征 → prepare_inputs_labels_for_multimodal() → 多模态嵌入 → 模型前向传播

7. 核心设计模式

7.1 组合模式

- StreamVLNForCausalLM 组合了语言模型和视觉编码器
- CombineDataset 组合多个数据源

7.2 模板方法模式

- preprocess() 函数根据不同模型版本选择不同的预处理方法
- train() 函数定义标准训练流程

7.3 策略模式

- 支持多种训练策略：LoRA、全参数微调、部分参数微调

159 - 支持多种注意力实现：sdpa、flash_attention_2等
160
161 8. 关键特性
162 8.1 SlowFast上下文建模
163 - Fast-streaming：滑动窗口KV缓存处理实时输入
164 - Slow-updating：Token pruning处理长序列历史记忆
165
166 8.2 多任务支持
167 - 统一的接口支持导航、问答、视觉字幕等任务
168 - 通过任务ID区分不同任务类型
169
170 8.3 流式生成
171 - 支持在线、多轮对话式的动作生成
172 - 维护跨时间步的上下文状态
173
174 这个架构设计体现了现代多模态AI系统的典型特征：模块化、可扩展、支持多种训练策略和部署场景。

llava模块分析

代码块

```
1 StreamVLN项目中llava目录的作用
2
3 1. 核心作用：多模态基础设施
4
5 llava目录在StreamVLN项目中扮演多模态基础架构的角色，提供了完整的多模态大模型基础设施，
StreamV
6 LN基于此构建了专门的语言视觉导航系统。
7
8 2. 主要功能模块
9
10 2.1 模型架构层 (llava/model/)
11
12 核心架构类：
13 - llava_arch.py：定义LLaVA元模型接口
14 - LlavaMetaModel：多模态元模型基类
15 - LlavaMetaForCausalLM：因果语言模型元接口
16
17 语言模型适配 (language_model/)：
18 - llava_qwen.py：Qwen模型的多模态扩展
19 - llava_llama.py：LLaMA模型的多模态扩展
20 - llava_mistral.py：Mistral模型的多模态扩展
21 - llava_mixtral.py：Mixtral模型的多模态扩展
22
23 视觉编码器 (multimodal_encoder/)：
```

- clip_encoder.py: CLIP视觉编码器
- siglip_encoder.py: SigLIP视觉编码器
- eva_clip_encoder.py: EVA-CLIP视觉编码器
- open_clip_encoder.py: OpenCLIP编码器
- imagebind.py: ImageBind多模态编码器

视觉重采样器 (multimodal_resampler/):

- spatial_pool.py: 空间池化重采样
- qformer.py: Q-Former重采样器
- perceiver.py: Perceiver重采样器

视觉投影器 (multimodal_projector/):

- pooler_projector.py: 池化投影器
- 将视觉特征投影到语言模型空间

2.2 训练基础设施 (llava/train/)

训练器:

- llava_trainer.py: LLaVA专用训练器 (StreamVLN使用)
- train.py: 标准训练脚本
- train_mem.py: 内存高效训练
- train_dpo.py: 直接偏好优化训练

2.3 工具和实用功能 (llava/)

核心工具:

- mm_utils.py: 多模态处理工具
- conversation.py: 对话格式化
- constants.py: 常量定义
- utils.py: 通用工具函数

3. 在StreamVLN中的具体应用

3.1 模型继承关系

StreamVLN继承链

StreamVLNModel —► LlavaQwenModel —► LlavaMetaModel + Qwen2Model
StreamVLNForCausalLM —► LlavaMetaForCausalLM + Qwen2ForCausalLM

3.2 关键功能借用

视觉编码能力:

使用LLaVA的视觉编码基础设施

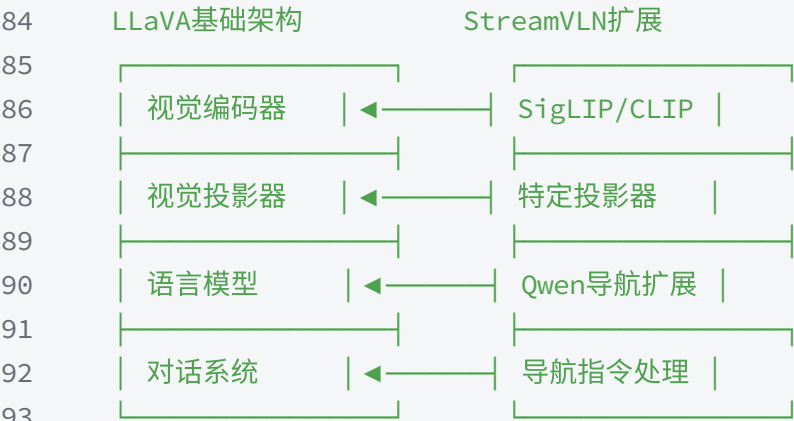
```
vision_tower = self.get_model().get_vision_tower()  
image_features = vision_tower(images)
```

多模态投影:

```
71 # 使用LLaVA的视觉投影器
72 mm_projector = self.get_model().mm_projector
73 projected_features = mm_projector(image_features)
74
75 训练基础设施：
76 # 使用LLaVA的训练器
77 from llava.train.llava_trainer import LLaVATrainer
78 trainer = LLaVATrainer(model=model, ...)
```

80 4. 架构设计优势

82 4.1 模块化设计



95 4.2 可扩展性

- 97 - 视觉编码器：支持多种预训练模型（CLIP、SigLIP、EVA-CLIP等）
- 98 - 语言模型：支持多种LLM（Qwen、LLaMA、Mistral等）
- 99 - 训练策略：支持多种训练范式（全参数、LoRA、DPO等）

101 4.3 研究价值

- 103 - 最新进展：集成了多模态大模型的前沿技术
- 104 - 标准化：提供了标准的多模态处理流程
- 105 - 可复现：基于成熟的LLaVA架构

107 5. 与StreamVLN的深度集成

109 5.1 视觉处理流水线

```
111 # LLaVA提供的视觉处理
112 image → vision_tower → vision_resampler → mm_projector → 语言模型
```

114 5.2 训练流程集成

```
116 # 使用LLaVA的训练基础设施
117 LLaVATrainer → 多模态数据加载 → 梯度更新 → 模型保存
```

118	
119	5.3 推理优化
120	
121	- KV缓存管理
122	- 批量推理支持
123	- 内存优化
124	
125	6. 代码组织价值
126	
127	6.1 代码复用
128	
129	- 避免重复造轮子
130	- 基于成熟的多模态框架
131	- 专注于VLN特定的创新
132	
133	6.2 维护性
134	
135	- 清晰的模块边界
136	- 标准化的接口
137	- 易于扩展和修改
138	
139	6.3 社区支持
140	
141	- 基于活跃的LLaVA项目
142	- 持续的技术更新
143	- 丰富的预训练模型
144	
145	总结
146	
147	llava目录在StreamVLN项目中扮演多模态基础架构提供者的角色，它：
148	
149	1. 提供核心基础设施：视觉编码、多模态融合、训练框架
150	2. 实现模块化设计：可插拔的视觉编码器和语言模型
151	3. 支持快速开发：基于成熟的LLaVA架构，专注于VLN创新
152	4. 确保标准化：遵循多模态大模型的标准实践
153	
154	这种设计让StreamVLN能够专注于导航任务特定的创新，而不需要重新实现基础的多模态处理能力。

StreamVLN模块分析

代码块

```
1 StreamVLN 目录模块化架构分析
2 整体架构概览
3 streamvln/
4 |—— model/ # 核心模型模块
```



```
5  |— dataset/                # 数据处理模块
6  |— utils/                 # 工具函数模块
7  |— habitat_extensions/    # 环境扩展模块
8  |— 核心训练/推理脚本
9  |— 配置文件
10 1. 核心模型模块 (model/)
11 主要文件: stream_video_vln.py
12 核心类:
13 StreamVLNModel: 继承自 LlavaQwenModel, 负责多模态特征融合
14 StreamVLNForCausalLM: 主要的模型类, 结合了 Qwen2ForCausalLM 和
    LlavaMetaForCausalLM
15 模块功能:
16 视频-语言-动作建模: 实现视觉、语言和动作的联合建模
17 记忆机制: 支持历史对话和长期记忆
18 多模态融合: 集成视觉编码器 (SigLIP) 和语言模型 (Qwen2)
19 流式处理: 支持在线实时推理
20 设计特点:
21 基于 transformer 架构
22 支持多轮对话式导航
23 可配置的可训练组件
24 2. 数据处理模块 (dataset/)
25 2.1 VLN 动作数据集 vln_action_dataset.py
26 功能:
27 处理 VLN 轨迹数据 (R2R、RxR、EnvDrop)
28 支持多模态输入: 视频帧、指令、动作序列
29 实现数据增强和预处理
30 核心组件:
31 _add_speaker_and_signal(): 对话格式化
32 preprocess_multimodal(): 多模态数据预处理
33 VLNActionDataset: 主数据集类
34 2.2 MMC4 数据集 mmc4_dataset.py
35 功能:
36 处理大规模图文对齐数据 (MMC4)
37 支持多任务学习 (QA、图像描述等)
38 提供 co-training 数据支持
39 3. 工具函数模块 (utils/)
40 3.1 通用工具 utils.py
41 功能:
42 定义特殊 token (<image>, <memory>, <video>)
43 提供通用辅助函数
44 数据格式转换和处理
45 3.2 分布式训练 dist.py
46 功能:
47 支持多 GPU 和多节点训练
48 提供分布式通信工具
49 梯度同步和模型同步
50 4. 环境扩展模块 (habitat_extensions/)
```

51 **4.1 地图模块** maps.py
52 **功能:**
53 扩展 Habitat-sim 地图功能
54 支持导航相关的地图操作
55 **4.2 测量模块** measures.py
56 **功能:**
57 定义导航评估指标
58 计算路径长度、成功率等指标
59 **5. 核心训练/推理脚本**
60 **5.1 训练脚本** streamvln_train.py
61 **功能:**
62 实现完整的训练流程
63 支持多阶段训练 (预训练 + 微调)
64 集成 DeepSpeed 和混合精度训练
65 **核心流程:**
66 参数解析和环境设置
67 模型和分词器初始化
68 数据模块创建
69 训练器配置和启动
70 模型保存和检查点管理
71 **5.2 评估脚本** streamvln_eval.py
72 **功能:**
73 模型性能评估
74 支持多 GPU 评估
75 生成评估报告
76 **5.3 Dagger 收集** streamvln_dagger.py
77 **功能:**
78 实现数据集增强 (Dataset Aggregation)
79 在线数据收集和模型改进
80 支持多种数据集 (R2R、RxR、EnvDrop)
81 **5.4 智能体** streamvln_agent.py
82 **功能:**
83 实现导航智能体
84 集成 Habitat-sim 环境
85 支持实时决策和动作执行
86 **核心类:**
87 **VLNEvaluator:** 评估器, 处理环境交互和模型推理
88 **5.5 轨迹生成** streamvln_trajectory_generation.py
89 **功能:**
90 生成预训练轨迹数据
91 支持大规模轨迹生成
92 **5.6 实时服务器** http_realworld_server.py
93 **功能:**
94 支持真实机器人部署
95 提供 HTTP API 接口
96 **6. 配置模块**
97 **参数配置** args.py

98 **核心类：**
99 **ModelArguments：** 模型相关参数
100 **DataArguments：** 数据相关参数
101 **TrainingArguments：** 训练相关参数
102 **功能：**
103 统一管理所有超参数
104 支持灵活的模型配置
105 便于实验管理和参数调优